Multi-Process-Parallel-File-Keyword-Searcher

CSC209

Kaiwen Yang & Ziheng Wang & Yifei yang

Project Overview

Our project is about Category 1: Multi-Process Application Using Pipes. This project looks for keywords in lots of text files at the time. Users give the project a keyword and a list of filenames. The project creates one parent process and three worker processes. The parent process creates the worker processes. It tells them what files to work on using a pipe and gets the results from them using another pipe. Each worker process gets a filename and a keyword. Then it scans the whole file and counts how many times the keyword is in it. After that, it sends the result back to the parent process. When all the files are done the parent process shows us how matches are found in each file. Then it provides a summary that says how many files were looked at, how many files had matches and how many matches there were in total. It is more efficient than doing things one by one.

Build Instructions

The main part of this project is to develop a multiprocess application that implements pipes to communicate between processes while searching keywords in multiple text files. The system is designed to split the work through three worker

processes to provide synchronous searching instead of independence, which can improve the efficiency and benefit from parallel processing.

On the other hand, we use a parent process to manage those workers correctly. By using a fork to create those workers, assign workers differently through pipes and collect the value returned from those workers, the system exhibits a structured process to process collaboration in a multiprocess environment.

Communication Protocol

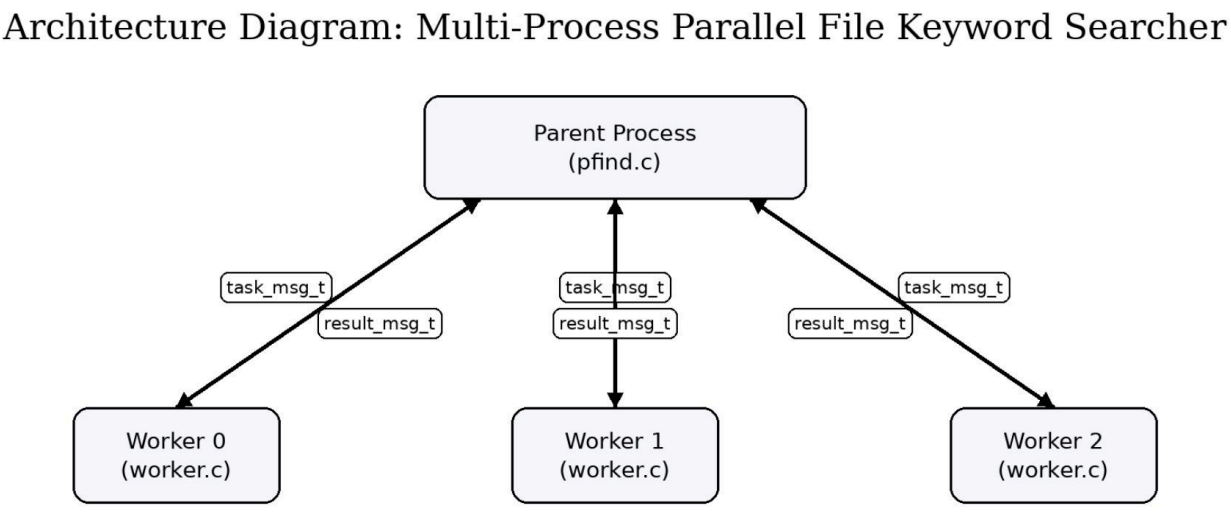
The program uses pipes to communicate between the process and three worker processes. There are two types of messages. Task messages are the messages that the main process sends to a worker and result messages that a worker sends back to the process. To make sure communication works well, when the parent sends a task to a worker, the message includes the filename and the keyword to be searched. This message is encoded in a fixed-size format. Then the receiver always knows the location to put the data and amounts of bytes to read. It also helps the receiver know how to understand the data. The worker will store the received data in a predefined structure, which makes it easier to separate the filename from the keyword and read the meaning of the message correctly. Then, by sending back the result to the parent, the message with filename, number of matches and a status value that shows whether the task was completed successfully can efficiently return the information about the amount of bytes to read and the place to store it.

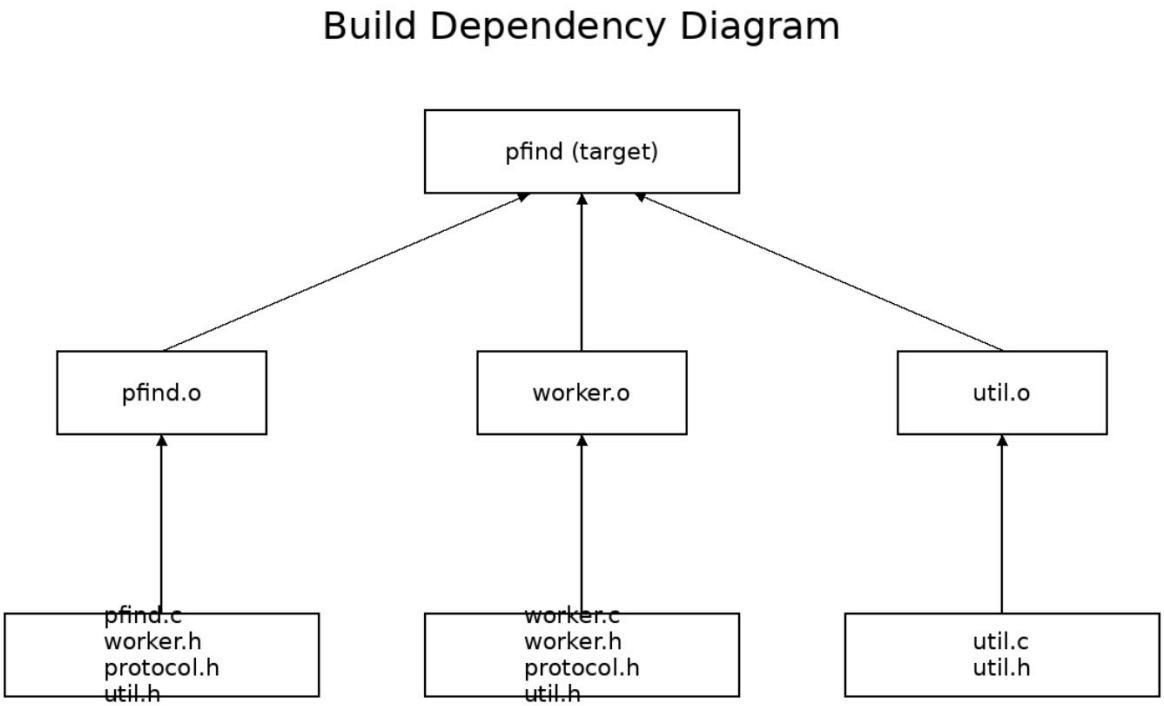
The filename identifies the processed file, the match count shows how many times the keyword appeared, and the status field tells the parent whether the result is

valid. The receiver in each case must read the complete message before using it, because a partial read could lead to incorrect processing or invalid output. Therefore, the semantic is also clear.

For the error handling part, if a process reads fewer bytes than expected, the message is treated as incomplete and the program will process incorrectly. Also, if a worker can't open a file or encounters other problems, it will send a special error message to the parent so the parent can take the issue instead of waiting forever. This protocol ensures that communication between processes remains predictable and safe throughout the program.

Architecture Diagram





Cocurrency

Our application uses concurrency through a parent-worker process model and is not over complicated. At the beginning, the parent process first sets up the communication pipes and then forks the worker processes before any other searchings and sendings begin. Each child process inherits the appropriate pipe ends and enters a waiting state. At this stage, child pipes are ready to receive tasks from the parent. The parent determines that workers are ready once all forks have completed successfully and the children are blocked waiting for messages sent from the parent. While executing this program, the parent will start assigning search tasks to the workers. After each worker processes its assigned task, it will send the result back through the pipe. So, synchronization is handled by the message-passing mechanism itself rather than by a more complicated concurrency design, because communication happens through

blocking reads and writes pipe ends. Then, all the work will be completed and then parent closes the write pipe ends and collects every child process using waitpid, ensuring that all workers terminate cleanly and that no zombie processes are left behind.

Error Handling and Robustness

In this project, we designed our architecture so system environments may be unpredictable. In order to prevent system crashes, we implemented some defensive strategies.

Firstly, we use a while loop to iterate the exactly number of bytes that we wanted, so we can make sure that the function will only return when the exactly require size is met. In case of signal interruptions that might disturb the read or write operations, our read_full and write_full implement a safe check when errno == EINTR, and use a continue to restart the operation. This can ensure our pipeline to execute without easily breaking. On the other hand, While a worker may be writing to a closed pipe or the parent was reading on a closed pipe, we return the bytes we've read before and loops into the else branch in pfind.c to kill and free the workers.

Meanwhile, we implement a load balancing function to ensure that our program can handle exhaustive tasks and work properly. While some worker is dealing with a giant file, a multiplexing dispatcher uses select to find some workers that are free and enactive at that time to guarantee the utilizations.

Team Contribution

The project was developed collaboratively by three team members. There are parts that we separate the missions and parts done together：

- Kaiwen Yang (Parent Process & Task Management):

    - Designed and implemented the parent process logic in pfind.c

    - Handled worker creation using fork() and pipe()

    - Managed task distribution to workers

    - Implemented result collection and output formatting

    - Coordinated job scheduling and termination of workers

- Yifei Yang (Worker Implementation & File Processing):

    - Implemented the worker logic in worker.c

    - Developed the run_worker() function to continuously process incoming tasks

    - Implemented keyword searching functionality using file I/O (fopen, fgets) and string matching (strstr)

    - Ensured correct counting of keyword occurrences, including multiple matches per line

    - Constructed result messages and sent them back to the parent process

    - Handled file-related errors and communication robustness

- Jerry Wang (Communication Utilities & Protocol Design):

    - Designed the communication protocol (protocol.h)

    - Defined task_msg_t and result_msg_t structures

    - Implemented read_full() and write_full() functions in util.c to ensure reliable data transmission

    - Assisted in debugging inter-process communication issues

    - Contributed to testing and validation of the system

- All members contributed to testing, debugging, and integration of the final system. We also separate this report into sections and each is responsible for different sections. We prepare and record the video together and introduce our own part.